

Name: Marcelo Salvador

Date: Mar 2, 2026

Topic: Memory Facts (Python + AI)

MEMORY FACTS (Python + AI)

1. Large lists consume memory quickly

Why:

- Python lists store references (8 bytes each on 64-bit)
- Each Python object has overhead (~28 bytes for int)
- Millions of items explode memory

Example:

```
import sys

lst = list(range(1_000_000))
print(sys.getsizeof(lst))
```

But that only shows container size — not element objects.

In AI:

- Raw Python lists are inefficient for numeric workloads
- Use NumPy arrays (contiguous memory, C-backed)

2. NumPy arrays are contiguous memory

code

```
List:
[ ref ][ ref ][ ref ][ ref ]
```

```
NumPy:
| 1 | 2 | 3 | 4 |
```

Contiguous memory:

- Better CPU cache usage
- SIMD vectorization
- Faster linear algebra

In training:

- Batch matrix operations dominate runtime
- Memory layout directly affects throughput

3. Copying data slows training

Hidden copies:

```
python
b = a[:]      # full copy
b = a.copy()  # full copy
```

In NumPy:

```
python
b = a.view()   # no copy
b = a.copy()   # full copy
```

AI implication:

- Extra copies double memory usage
- Copying large tensors increases training latency

4. Mutability errors cause silent bugs

Classic issue:

```
python
def add(x, lst=[]):
    lst.append(x)
    return lst
```

AI example:

- Shared mutable preprocessing buffer

- Shared global model state
- Accidental dataset mutation

These bugs:

- Don't crash
- Corrupt training silently
- Are extremely hard to detect

COMPUTATIONAL FACTS

5. Time complexity dominates scaling

Training loop:

```
python  
  
for epoch in range(E):  
    for batch in data:    # O(n)  
        update()
```

If your preprocessing is $O(n^2)$, training explodes.

AI rule:

- Everything must be $O(n)$ or $O(n \log n)$
- Avoid nested loops over dataset

6. Recursion is dangerous in AI workloads

- Python recursion limit ≈ 1000 .
- Dynamic programming $>$ naive recursion.
- Never use deep recursion in training pipelines.

7. Vectorization beats loops

Bad:

```
python

for i in range(len(arr)):
    arr[i] *= 2
```

Good:

```
python

arr *= 2
```

Vectorization:

- Uses C backend
- Avoids Python interpreter overhead
- Critical for large-scale training

CONCURRENCY FACTS

8. GIL limits CPU-bound multithreading

Python's GIL:

- Only one thread executes Python bytecode at a time.

AI implications:

- CPU-bound preprocessing → use multiprocessing
- GPU-bound work → not limited by GIL

9. Race conditions corrupt models

If multiple threads update model weights:

- Non-deterministic behavior
- Reproducibility breaks
- Subtle statistical drift

Solution:

- Locking

- Stateless serving
- Atomic updates

DATA INTEGRITY FACTS

10. Garbage in → garbage out

AI systems amplify bad data.

Bad labels:

- Bias models
- Reduce generalization
- Create fairness issues

Always enforce:

- Schema validation
 - Range validation
 - Null handling
 - Duplicate detection
-

11. Train/test leakage is catastrophic

If training sees test data:

- Accuracy artificially high
- Model fails in production

Prevent:

- Strict data splitting
 - Immutable test sets
 - Hash-based splits
-

STATISTICAL FACTS

12. Overfitting is inevitable without control

Symptoms:

- Training loss ↓
- Validation loss ↑

Solutions:

- Regularization
 - Dropout
 - Early stopping
 - Cross-validation
-

13. Models are probabilistic, not certain

Predictions are:

- Conditional probabilities
- Not truth

Engineering implication:

- Add confidence thresholds
 - Reject uncertain predictions
 - Log probability distributions
-

SYSTEM DESIGN FACTS

14. AI systems must be observable

Monitor:

- Prediction drift
- Feature distribution shift
- Latency (p95, p99)
- Error rate

Without monitoring:

- Silent degradation

15. Determinism matters

Set seeds:

```
python

import random
import numpy as np

random.seed(42)
np.random.seed(42)
```

Otherwise:

- Hard to reproduce results
- Debugging becomes guesswork

OPTIMIZATION FACTS

16. Learning rate is everything

Too high:

- Divergence

Too low:

- Slow convergence

Use:

- Schedulers
- Warmup
- Decay

17. Exploding gradients destroy training

Solutions:

- Gradient clipping
- Normalization layers
- Proper initialization

ETHICAL + SAFETY FACTS

18. Bias propagates exponentially

Small dataset bias → amplified by model.

Mitigation:

- Stratified sampling
- Fairness audits
- Calibration checks

19. Models degrade over time

Data drift:

- Customer behavior changes
- Market changes
- Policy changes

Need:

- Retraining pipeline
- Drift detection
- Performance alarms

STORAGE FACTS

20. Serialization must be stable

Pickle is unsafe for untrusted sources.

Better:

- Torch save (controlled)
- ONNX for portability
- Versioned model registry

ARCHITECTURAL FACTS

21. Separate concerns

Pipeline layers:

code

```
Raw Data
↓
Validation
↓
Transformation
↓
Feature Store
↓
Training
↓
Model Registry
↓
Serving
↓
Monitoring
```

Never mix:

- Training logic
- Serving logic
- Data ingestion

CATEGORY MISTAKE FACTS

Common AI mistakes:

- Using list instead of set (duplicates allowed)
- Using float for money
- Using local time instead of UTC
- Comparing floats directly (==)

FLOAT FACT

python

```
0.1 + 0.2 != 0.3
```

In AI:

- Never compare floats directly
- Use tolerance:

```
python
```

```
abs(a - b) < 1e-8
```

PRODUCTION FACTS

22. Model size impacts latency

Large models:

- Higher memory footprint
- Longer inference time
- Higher cost

Tradeoffs matter.